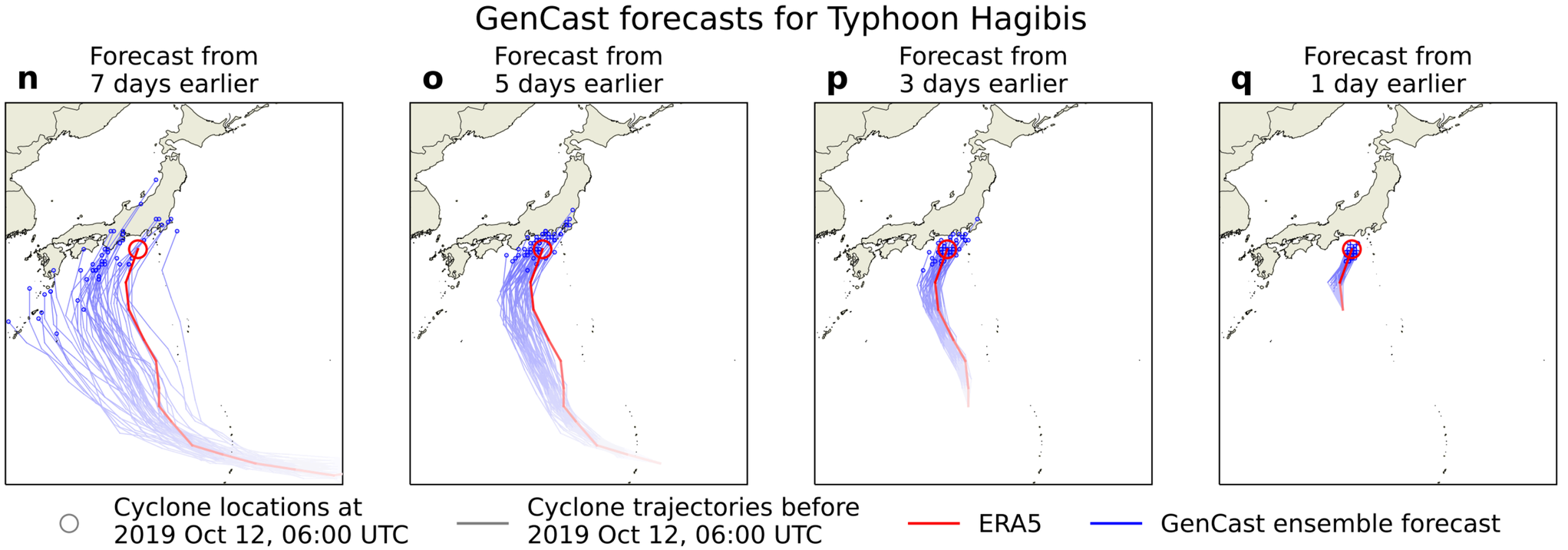
A Splitting Method for SDE Terminal-Law Estimation

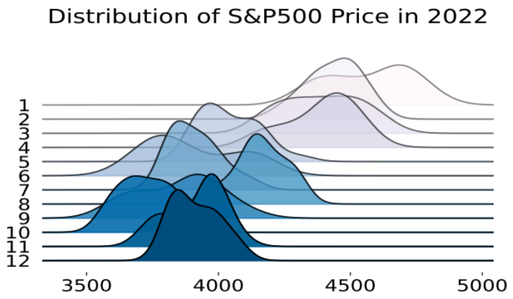
Rushil Gupta

We like to make predictions

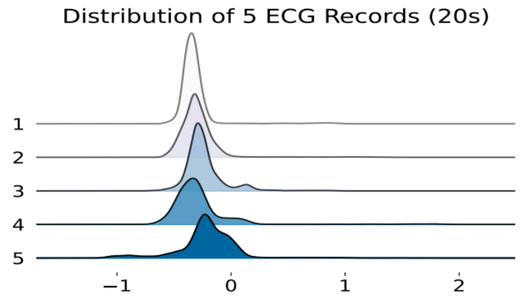


GenCast predicting **weather** in the near future

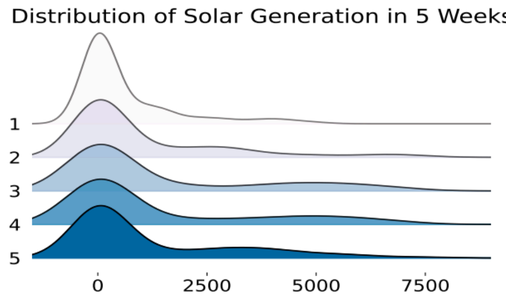
We like to make predictions



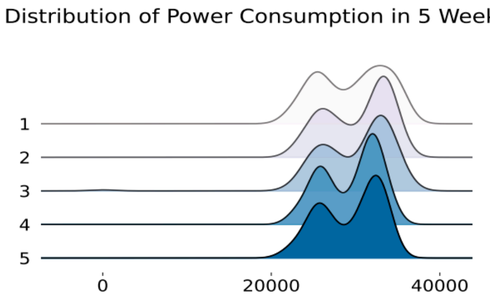
(a) S&P500 Monthly



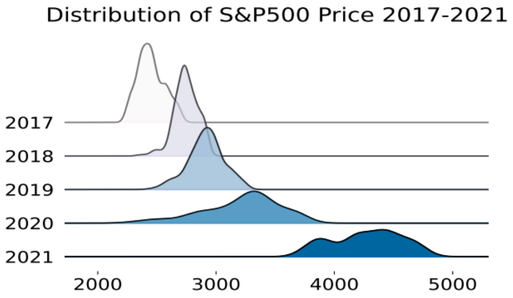
(b) ECG 20s



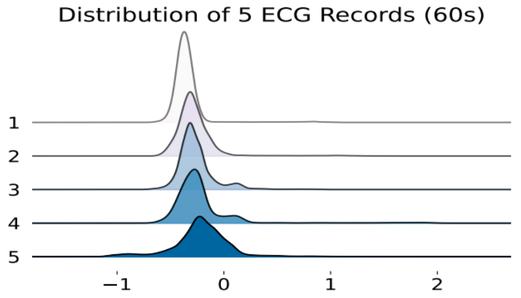
(c) Solar Weekly



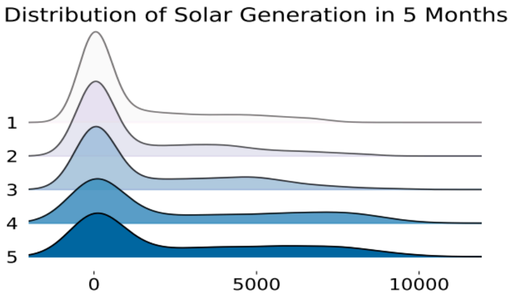
(d) Power Weekly



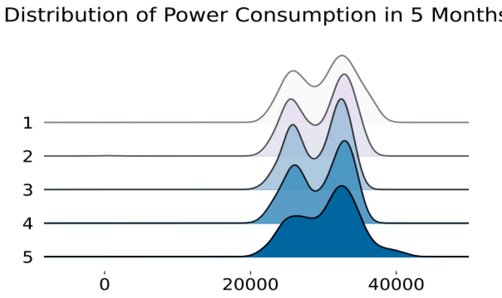
(e) S&P500 Yearly



(f) ECG 60s



(g) Solar Monthly



(h) Power Monthly

Predicting the **distribution** of **asset prices**

Generally, we are interested in…

Estimating the **probability distribution** of weather, asset prices, etc.

Many models of interest use **stochastic differential equations** (SDEs)

Often with a **budget** constraint - fixed compute time / trading window

Stochastic Differential Equations

An stochastic differential equation (SDE) is:

$$\mathrm{d}X_t = \mu(t, X_t)\,\mathrm{d}t + \sigma(t, X_t)\,\mathrm{d}B_t$$

$\mu(t, X_t)\,\mathrm{d}t$: Drift

$\sigma(t, X_t)\,\mathrm{d}B_t$: Diffusion

$\mathrm{d}B_t$ → Brownian Motion

We solve by discretising

$$X_{t_{i+1}} = X_{t_i} + \mu(t_i, X_{t_i})\,\Delta t_i + \sigma(t_i, X_{t_i})\,\Delta B_i, \quad 0 = t_0 < \cdots < t_m = T$$

We take m steps.
Each step costs 1 unit

Our goal

Estimate the terminal distribution function

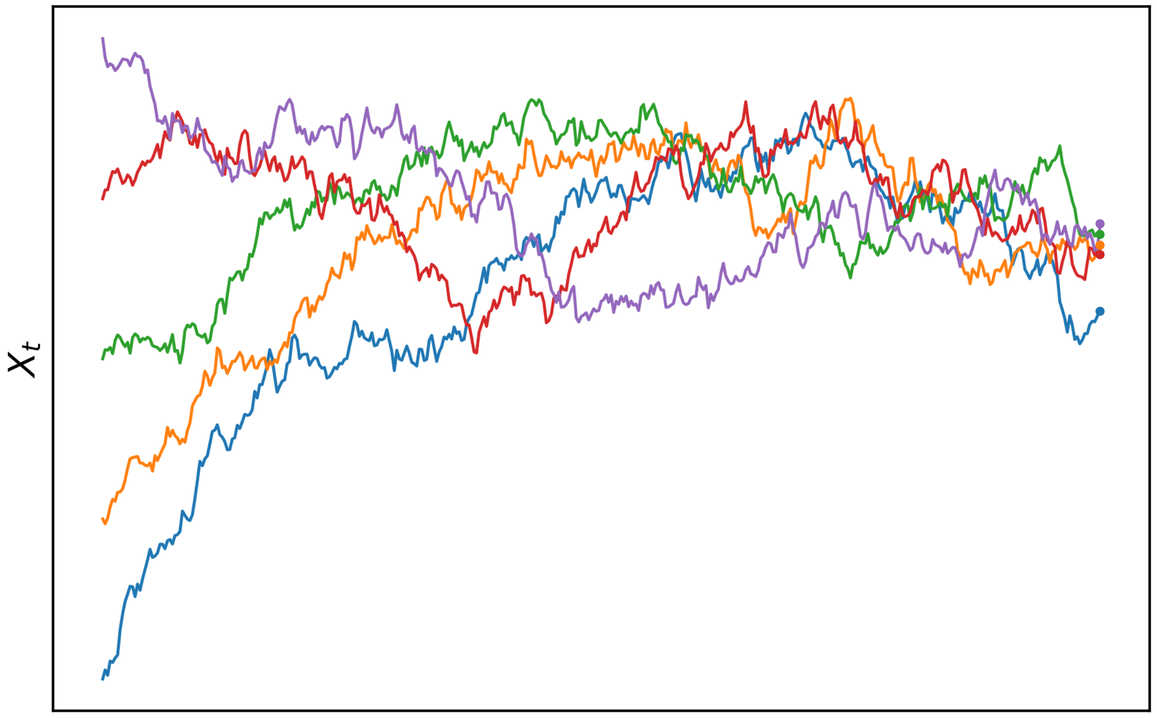
$$\widehat{F}(x) \approx F(x) = \mathbb{P}(X_{t_m} \leq x), \quad \forall x$$

Under the budget constraint

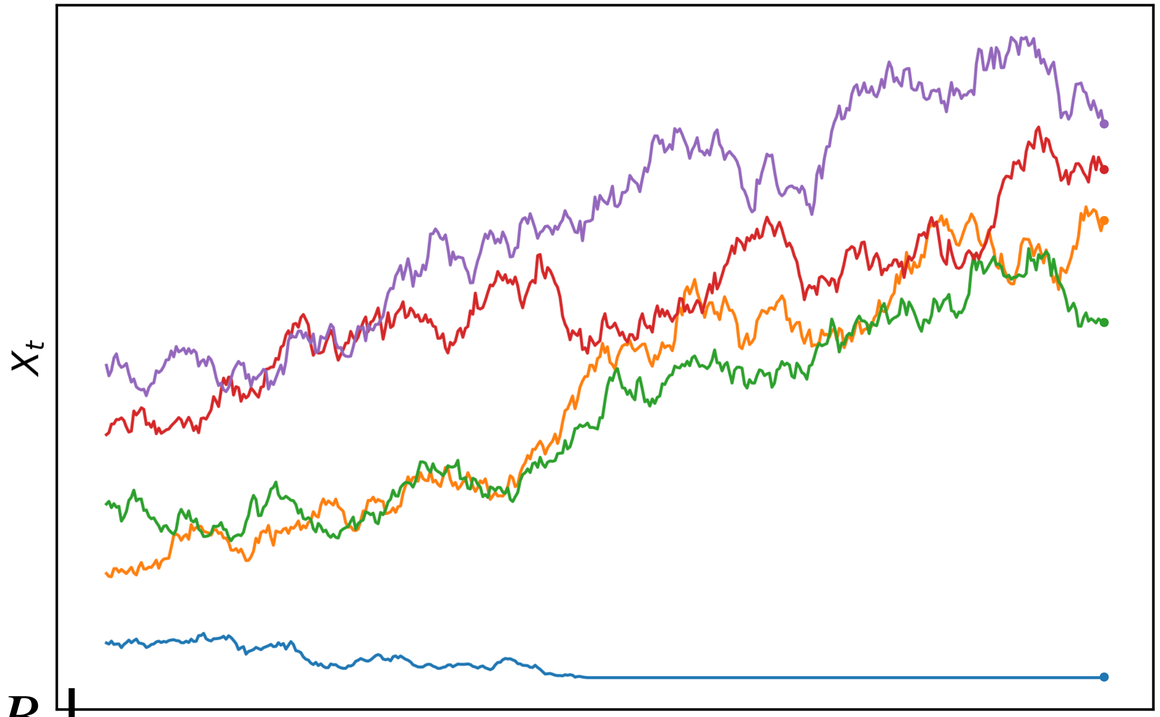
"Total transitions" should be at most B

The usual method

Simulate as many independent paths as possible, each with m steps



$dX_t = (1 - X_t)\,dt + 0.25\,dW_t$

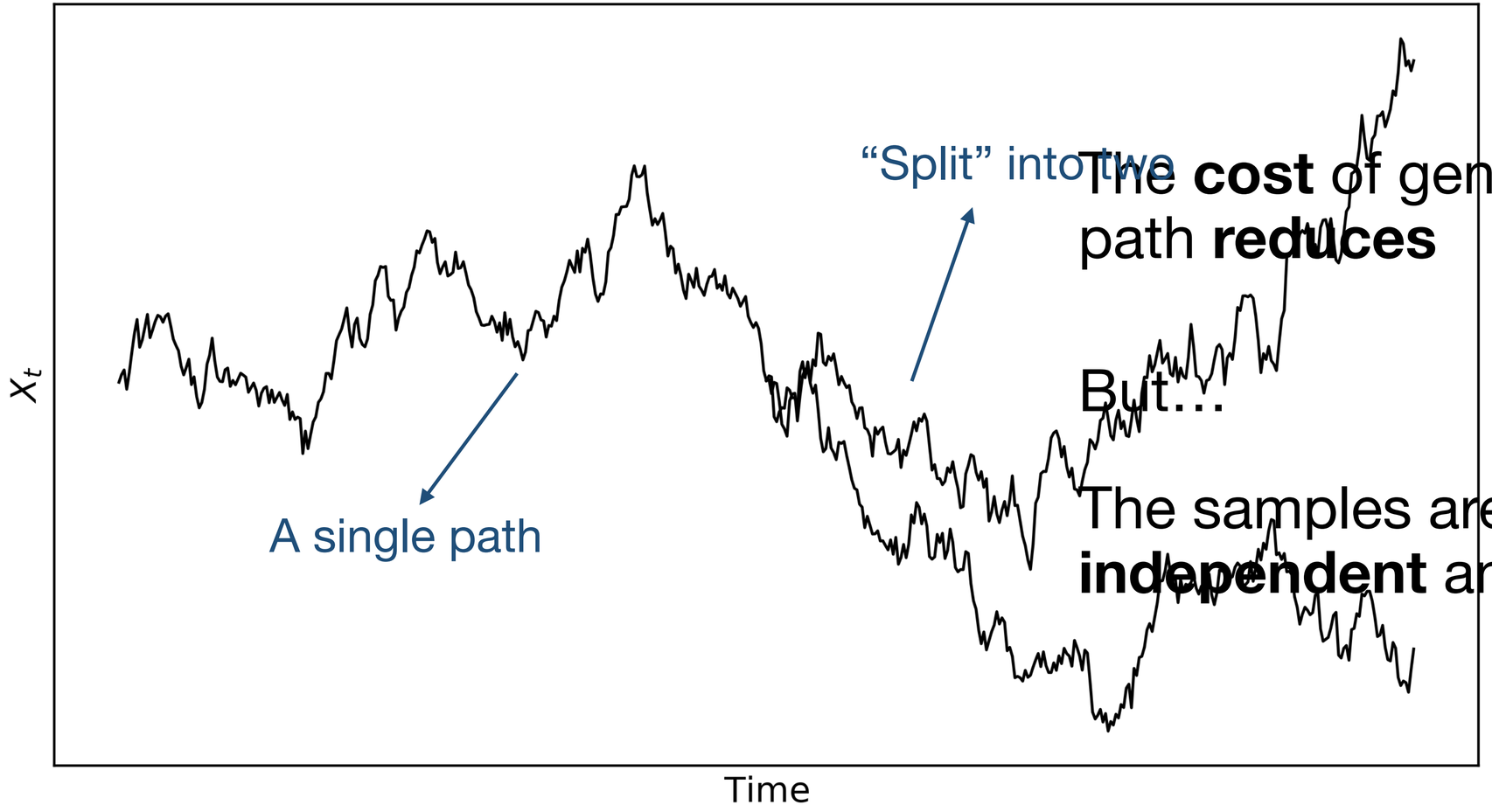


$dX_t = 0.1X_t\,dt + 0.3\sqrt{X_t}\,dW_t$

This allows us to simulate at most $n = \left\lfloor \frac{B}{m} \right\rfloor$ paths (500 steps above)

How about…

We “reuse” parts of a path



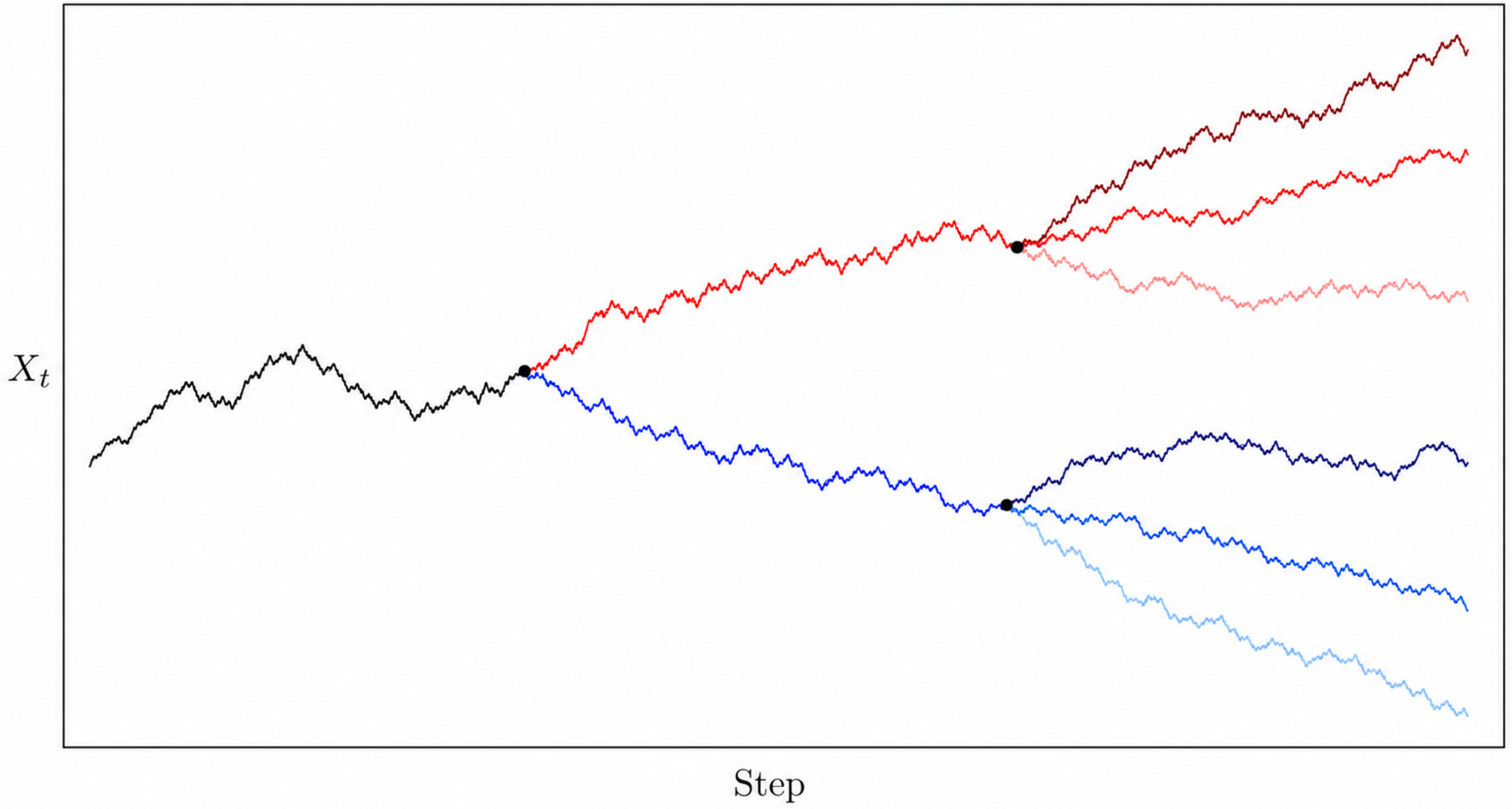
The **cost** of generating a path **reduces**

But…

The samples are **not independent** anymore

Why midpoint? Why split by two?

We can split at more times and by different amounts to make a “tree”



The Question

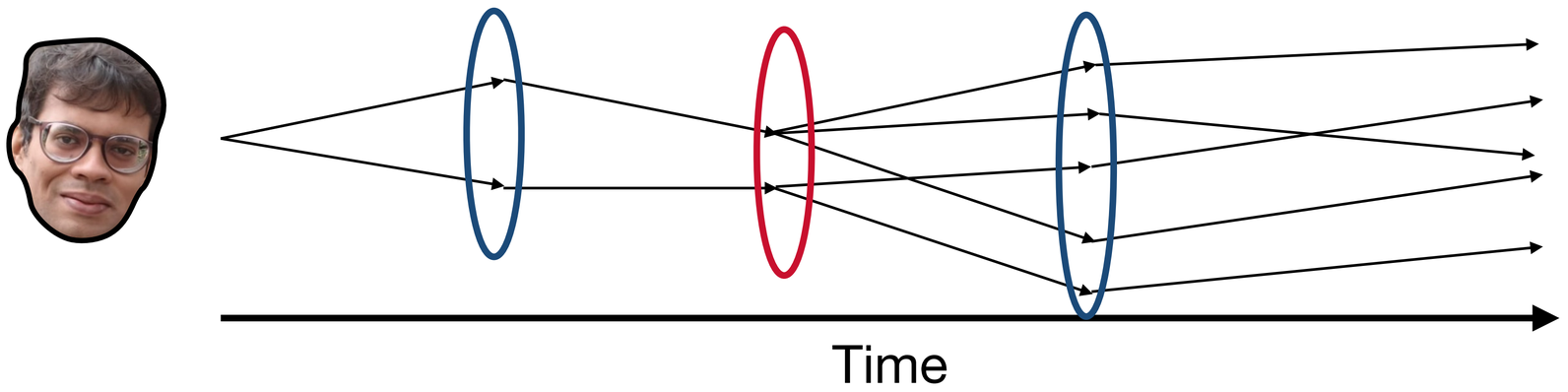
How do find the “optimal” tree which maximises **bang for the buck**

How much to split at each time point to minimise the error in estimating the distribution function?

The Answer

Split highly in time periods where the SDE is the most **uncertain**

Example: On a Thursday night, Ojas is the SDE and follows the paths below. Whenever there is an option, he takes a **random** path. We want to know where he ends



No options - “certain”
Splitting does not help

Many options - “uncertain”
Split Here

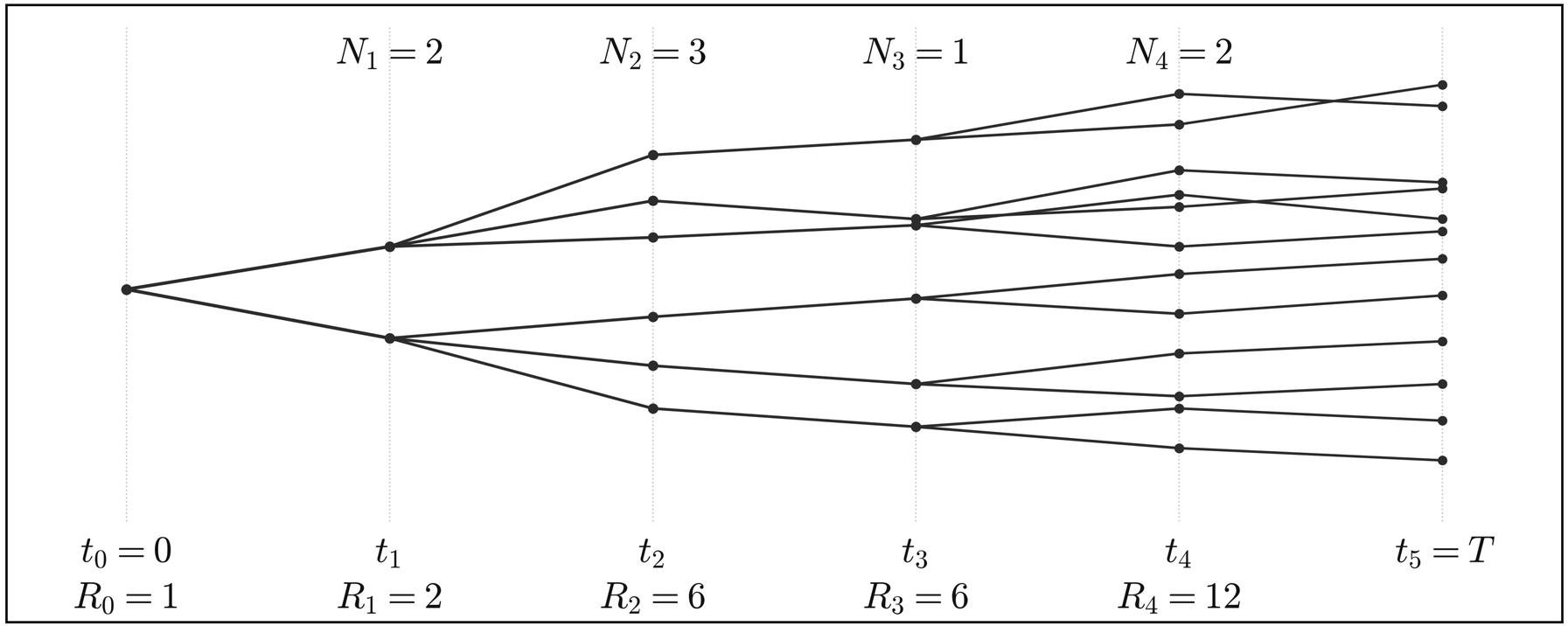
Let’s Get Formal

X_{t_i} : State at time t_i

N_i : Amount to "split" at time t_i

$\hat{F}_R$: CDF estimate using "strategy" R

$R_i = \Pi_{j=1}^{i} N_j$: Total splits till time t_i



Let’s Get Formal

The notion of error we use is the Kolmogorov-Smirnov distance

$$\mathrm{KS}\left(\hat{F}_R, F\right) = \sup_x \left|\hat{F}_R(x) - F(x)\right|$$

The optimisation problem

We want to solve

$$\min_{R} \max_{x} \left| \hat{F}_R(x) - F(x) \right|$$

$$\text{subject to} \quad n \sum_{i=0}^{n} R_i = B,$$

$$R_{i+1} \geq R_i \quad \forall i$$

But this is generally intractable



Result 1: A “close” and tractable objective

We lower and upper bound the expected objective function, and derive a closely related objective

$$\min_{R} \max_{x} \operatorname{Var}\left(\hat{F}_R(x)\right)$$
$$\text{subject to} \quad n\sum_{i=0}^{n} R_i = B, \qquad R_{i+1} \geq R_i \quad \forall i$$

It is enough to look at the variance alone

All the dependence can be understood just by the variance

Result 2: Variance Decomposition

To understand this better, we show that

$$\mathrm{Var}\left(\hat{F}_R(x)\right) = \sum_{i=0}^{m-1} \frac{v_i(x)}{R_i}, \qquad v_i(x) = \mathbb{E}\left[\mathrm{Var}\left(\mathbb{P}\left[X_{t_m} \le x \mid X_{t_{i+1}}\right] \mid X_{t_i}\right)\right]$$

Variance “added” at step *i*

The **total variance** is the sum of the variance of each step

A proof uses the
law of total variance

Result 3: The solution (Simplified)

Let $(v_0, \cdots, v_{m-1})$ be the "largest" variance vector. Then,

$$R_i^* \propto \sqrt{v_i}\,.$$

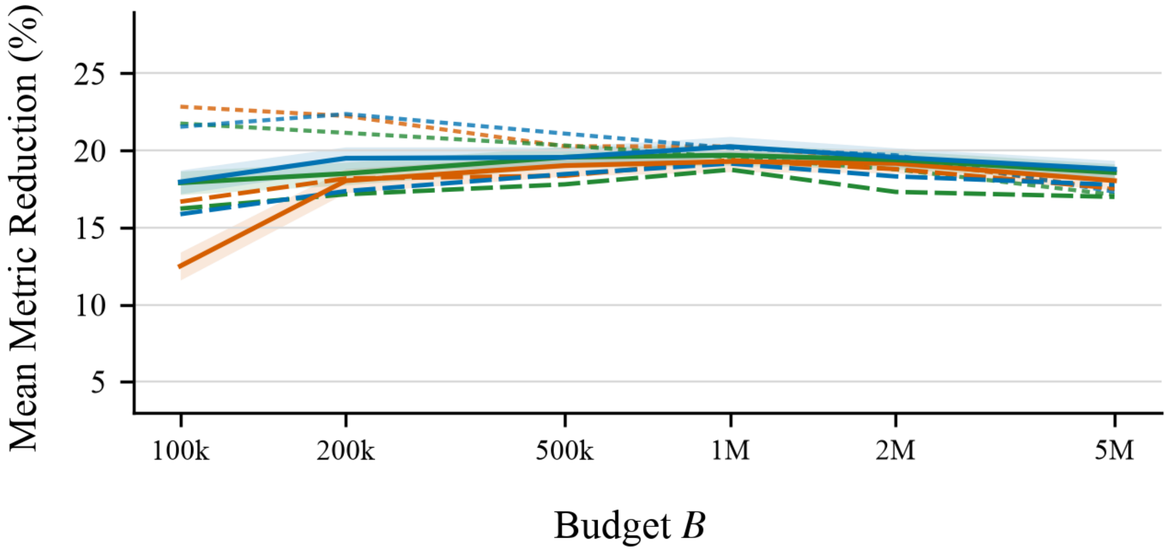
Here, "largest" means

$$v = \operatorname{argmax} \sum_{i=0}^{m-1} \sqrt{v_i}$$

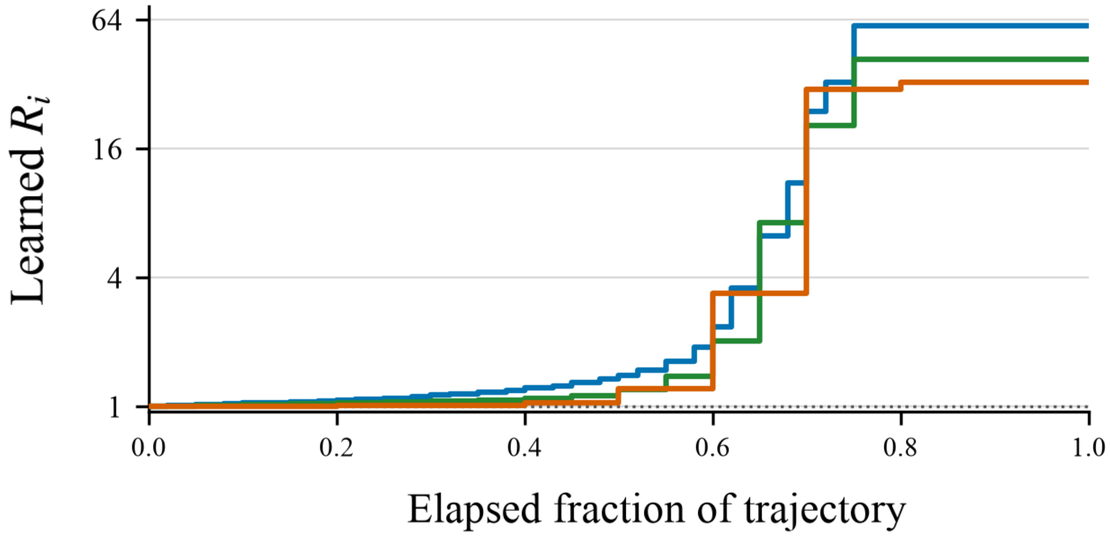
*Some more changes are needed for the monotonicity constraint

Does it work?

Yes! For a diffusion model, comparing against i.i.d. sampling, we get 20% reduction in our error. The allocations are also satisfying.



Error (KS) is reduced by 20%



Splits only near the midpoint

Thank you :)